

Blue Machines SDE-1 Assignment

Build Backchanneling for LiveKit and Prove Its Impact

Timebox

10-14 hours

We care more about your engineering decisions and reasoning than about implementing every possible feature.

Problem

Human listeners naturally acknowledge someone while they are speaking:

"mm-hmm"

"right"

"okay"

Voice agents usually stay silent until the user finishes, which can make long conversations feel unnatural.

Build a **backchanneling system on top of LiveKit Agents** that allows the AI to occasionally acknowledge the user while they are still speaking.

The difficult part is not playing `"mm-hmm"`.

The challenge is doing this **without breaking turn-taking or increasing the latency of the agent's actual response**.

What to Build

Your solution should include three things:

1. Backchannel Engine

Detect when it is appropriate for the agent to produce a short acknowledgement while the user is speaking.

Consider signals such as:

- how long the user has been speaking,
- interim transcripts,
- end-of-turn probability,

- agent state,
- recent backchannels,
- pauses,
- whether a normal response is about to begin.

You decide the policy.

The acknowledgement should not behave like a normal agent response or pollute the main LLM conversation context.

Do not modify LiveKit itself or rely on private LiveKit APIs.

2. Benchmark / Replay System

We want to compare:

A. Normal LiveKit agent

vs.

B. Same agent with your backchanneling enabled

Create repeatable test scenarios so the same user interaction can be run through both configurations.

Do not depend only on manually speaking into the microphone twice.

Create at least **6-8 scenarios**, including:

- short answer,
- long monologue,
- user approaching end-of-turn,
- pause in the middle of a sentence,
- fast speaker,
- noisy audio,
- long speech requiring multiple potential backchannels,
- user stopping just as a backchannel is about to play.

Run scenarios multiple times so results are not based on a single measurement.

Measure Latency

Instrumentation is an important part of the assignment.

At minimum, measure:

Actual response latency

User speech ends
↓
Agent's real response audio begins

Compare **P50** and **P95** with and without backchanneling.

This is the most important result.

Backchannel latency

Measure approximately:

Backchannel decision
↓
Backchannel becomes audible

Also capture useful pipeline timings where possible, such as:

- EOT detection,
- LLM TTFT,
- TTS first audio,
- STT finalization.

We should be able to answer:

Did adding backchanneling make the actual agent slower?

And if so:

Where did the extra latency come from?

Measure Behaviour

Latency alone is not enough.

Track useful quality signals such as:

- number of backchannels,
- backchannels per long user turn,
- acknowledgements too close to end-of-turn,
- overlap between agent and user,
- cancelled backchannels,
- backchannels that delayed a normal response.

You should define what you consider a **bad backchannel** and explain why.

UI — Required

Build a simple UI that lets us inspect the experiment.

It should show an overall comparison like:

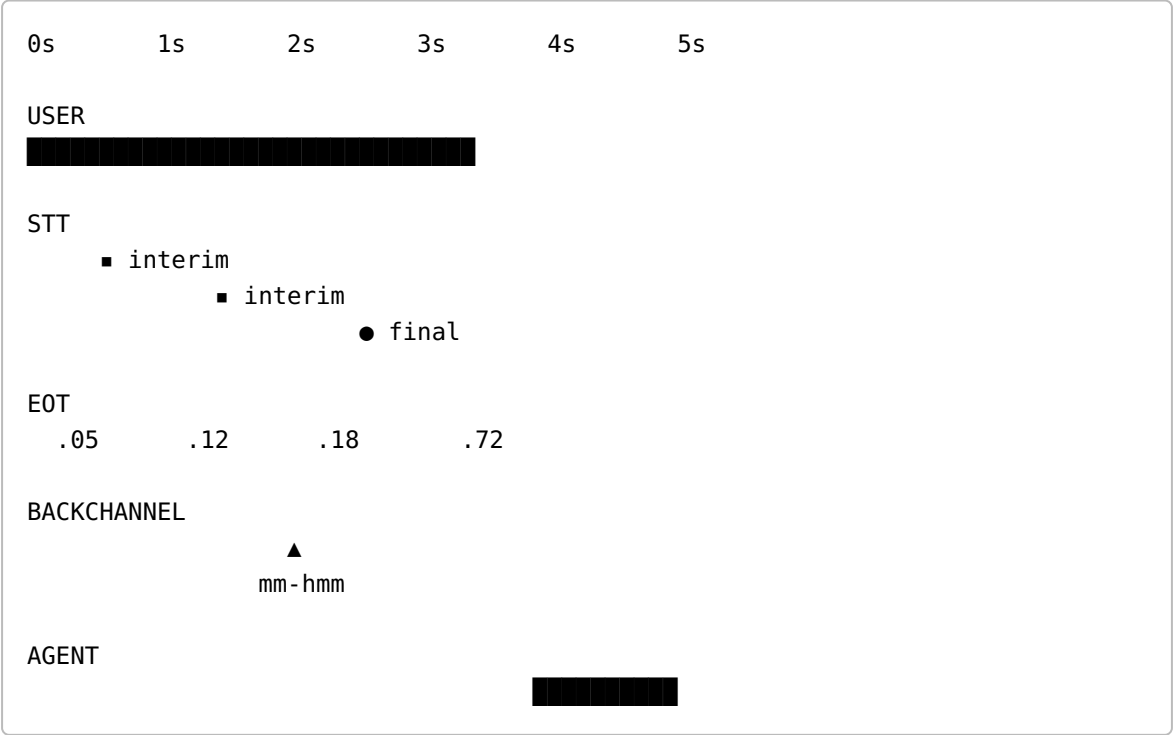
Metric	Baseline	Backchannel	Delta
Response P50	480 ms	498 ms	+18 ms
Response P95	710 ms	752 ms	+42 ms
LLM TTFT	145 ms	148 ms	+3 ms
EOT collisions	—	2	—

Use your actual numbers.

Conversation Timeline

For an individual run, we should be able to visually understand what happened.

For example:



The timeline should make it easy to see:

- user speech,

- transcript events,
- EOT signals,
- backchannel decisions,
- actual backchannel audio,
- user end-of-turn,
- real agent response.

We should also be able to compare a **baseline run** and a **backchannel-enabled run** for the same scenario.

Live Demo

Also provide a mode where we can talk to the agent live.

We should be able to enable/disable backchanneling and experience the difference.

Things We Expect You to Think About

We intentionally aren't specifying how to solve these.

Consider:

Race conditions

What happens if:

```
Backchannel selected
    ↓
TTS starts generating
    ↓
User finishes speaking
    ↓
Agent needs to answer
```

Does the acknowledgement still play?

Who has priority?

Cancellation

How do you cancel a backchannel that is no longer appropriate?

What happens if the user suddenly stops talking?

Async task management

How do you prevent every STT event or EOT event from creating uncontrolled async work?

Slow TTS

What happens if generating "mm-hmm" takes 500 ms?

Would pre-generated audio be better?

What trade-offs does that create?

End-of-turn prediction

How close to the expected end of a user's turn should backchannels be suppressed?

How would you determine that threshold?

Benchmark correctness

How do you know a 30 ms difference came from your system rather than normal network/provider variance?

How do you make the baseline-vs-experiment comparison fair?

Required Tests

At minimum cover:

- short speech → no backchannel,
- long speech → backchannel,
- cooldown / repeated acknowledgements,
- user stops before acknowledgement plays,
- approaching end-of-turn,
- agent starts its normal response,
- slow or failed TTS,
- rapid speaking / silence transitions,
- session shutdown.

We care about testing the behaviour of the system, not just utility functions.

Deliverables

Submit:

1. Source code
2. Working LiveKit demo
3. Benchmark scenarios
4. Benchmark results
5. Web UI
6. Automated tests
7. README

The README should explain:

- your architecture,
 - how the backchannel policy works,
 - important race conditions,
 - how latency is measured,
 - how you made the benchmark fair,
 - what became slower with backchanneling,
 - and what you would change before running this at production scale.
-

Evaluation

Real-time / async correctness — 25%

Benchmarking and latency analysis — 20%

Backchannel quality — 20%

UI and observability — 15%

Code design — 10%

Testing and engineering reasoning — 10%

Bonus

Pick any that interest you:

- Compare generated TTS vs cached backchannel audio.
- Support English + Hindi/Hinglish.
- Simulate network jitter / latency.
- Add configurable backchannel policies in the UI.
- Automatically flag latency regressions.
- Compare two different backchannel policies.

Key Question

At the end of the assignment, we should be able to look at your UI and answer:

Did backchanneling make the conversation feel more natural without making the actual agent slower or damaging turn-taking?

More importantly, your system should make it clear **why**.